

JAVA

Collections Framework

List, Set, Map e as coleções especializadas de Enum — da teoria de hashing às árvores balanceadas.

Apostila completa de referência, com exemplos práticos e explicações detalhadas de cada método do framework.

CONTEÚDO DA APOSTILA

01 Visão Geral do Framework

02 Set e HashSet

03 hashCode() e equals()

04 LinkedHashSet, TreeSet e NavigableSet

05 Operações de Set

06 Map, HashMap e Views

07 LinkedHashMap, TreeMap e NavigableMap

08 EnumSet e EnumMap

01

Hierarquia de interfaces, a classe Collections e o mapa de métodos essenciais

02

Unicidade, ausência de ordem garantida e operações em tempo constante

03

O contrato entre os dois métodos e o funcionamento interno do HashSet

04

Ordem de inserção, ordenação automática e navegação por proximidade

05

União, interseção, diferença e diferença simétrica entre coleções

06

Pares chave-valor, métodos modernos do JDK 8+ e as visões do Map

07

Ordem de inserção, cache LRU, chaves ordenadas e navegação por intervalo

08

Coleções especializadas para enums e representação por vetores de bits

01

CAPÍTULO

Visão Geral do Framework

Hierarquia de interfaces, a classe Collections e o mapa de métodos essenciais

Uma collection, em termos simples, é qualquer objeto capaz de agrupar outros objetos e disponibilizar formas padronizadas de percorrê-los, buscá-los ou modificá-los. O Java não trata cada estrutura de dados como um caso isolado: existe uma arquitetura única — o Collections Framework — que define contratos comuns (interfaces) e depois oferece várias implementações concretas para cada contrato, cada uma otimizada para um cenário diferente.

No topo da hierarquia está Iterable, que garante que qualquer collection pode ser percorrida com um for-each. Dela deriva Collection, que se ramifica em três famílias principais: List (mantém ordem de inserção e aceita duplicatas), Set (nunca aceita elementos repetidos) e Queue (pensada para processamento em fila). O Map fica de fora dessa árvore — ele não estende Collection porque trabalha com pares chave-valor, e não com elementos soltos, mas ainda assim é considerado parte do framework.

```
// Esqueleto da hierarquia:  
  
// Iterable -> Collection -> List -> ArrayList, LinkedList  
  
// -> Set -> HashSet, LinkedHashSet, TreeSet  
  
// -> Queue -> ArrayDeque, PriorityQueue  
  
// Map (à parte) -> HashMap, LinkedHashMap, TreeMap
```

COLLECTION NÃO É COLLECTIONS

É fácil confundir os dois nomes. **Collection** (sem "s" no final) é a interface-mãe de List, Set e Queue. **Collections** (com "s") é uma classe utilitária, cheia de métodos estáticos prontos para ordenar, embaralhar ou pesquisar dentro de uma collection já existente. Uma é contrato, a outra é caixa de ferramentas.

A classe utilitária Collections

Em vez de cada implementação reescrever sua própria lógica de ordenação ou busca, o Java concentra essas operações genéricas na classe Collections. Ela funciona sobre qualquer List, Set ou Map, sem se importar com a implementação escolhida por trás.

```
List<Integer> exemplaresAno = new ArrayList<>(List.of(2019, 2005, 2022, 1998));

Collections.sort(exemplaresAno); // ordena em ordem crescente

Collections.reverse(exemplaresAno); // inverte a ordem atual

int posicao = Collections.binarySearch(exemplaresAno, 2019); // busca binária
```

Panorama de métodos essenciais

A tabela a seguir reúne os métodos mais úteis de List, Collections e Comparator. Logo depois, cada grupo é detalhado com explicação e um exemplo próprio.

Estrutura	Método	Função
Arrays	Arrays.fill()	Preenche todas as posições do array com o mesmo valor
Collections	Collections.fill()	Substitui todos os elementos já existentes numa lista
Collections	Collections.nCopies()	Gera uma lista imutável com N cópias do mesmo objeto
Collections	Collections.addAll()	Adiciona vários elementos de uma vez (varargs)
Collections	Collections.copy()	Copia os elementos de uma lista de origem para uma de destino
List	list.addAll()	Incorpora todos os elementos de outra coleção
List	List.copyOf()	Gera uma cópia imutável da lista
Collections	Collections.shuffle()	Embaralha os elementos de forma aleatória
Collections	Collections.reverse()	Inverte a ordem atual dos elementos
List / Collections	sort()	Ordena a lista — prefira list.sort() ao antigo Collections.sort()
Comparator	comparing() / thenComparing()	Definem o critério principal e critérios de desempate
List	subList()	Extrai uma fatia da lista (início inclusivo, fim exclusivo)
List / Collections	containsAll() / disjoint()	Verificam elementos em comum entre duas coleções
Collections	binarySearch()	Busca binária — exige lista previamente ordenada
List	indexOf() / lastIndexOf()	Localizam a primeira ou a última ocorrência de um elemento
Collections	frequency() / max() / min()	Contam ocorrências ou retornam extremos da coleção
Collections	rotate() / swap()	Deslocam elementos circularmente ou trocam duas posições

Detalhamento — List

`List.addAll(outraLista)`

Incorpora, ao final da lista atual, todos os elementos de outra coleção. Útil para consolidar o acervo de duas prateleiras numa lista só de resultado de busca.

`List.copyOf(lista)`

Retorna uma cópia **imutável** da lista informada. Qualquer tentativa de alterar o resultado lança `UnsupportedOperationException` — ideal para expor um resultado de consulta sem permitir que quem recebeu mexa na fonte original.

`lista.subList(inicio, fim)`

Recorta uma fatia da lista, do índice inicial (incluído) até o final (excluído). O resultado é uma *view*: alterações nela refletem na lista original.

`lista.containsAll(outraColecao)`

Confirma se todos os elementos de outra coleção já estão presentes na lista — bom para checar se um pedido de reserva pode ser totalmente atendido pelo acervo disponível.

`lista.sort(comparador)`

Forma moderna de ordenação, disponível diretamente na lista desde o Java 8. É preferível ao antigo `Collections.sort(lista)` por não exigir a classe utilitária.

`lista.indexOf(elemento) / lastIndexOf(elemento)`

Retornam a posição da primeira ou da última ocorrência de um elemento; devolvem -1 quando ele não é encontrado.

Detalhamento — Comparator

`Comparator.comparing(Livro::titulo)`

Define o critério principal de comparação — aqui, ordenar por título.

`.thenComparing(Livro::anoPublicacao)`

Encadeia um segundo critério, usado apenas quando o primeiro empata. Combinado com o exemplo acima, ordena por título e, em caso de mesmo título, pelo ano de publicação.

```
Comparator<Livro> porTituloEAno =  
  
    Comparator.comparing(Livro::titulo)  
  
    .thenComparing(Livro::anoPublicacao);  
  
acervo.sort(porTituloEAno);
```

Detalhamento — Collections (classe utilitária)

`Collections.fill(lista, valor)`

Substitui todos os elementos já existentes pelo valor informado — não adiciona posições novas, a lista precisa já ter o tamanho necessário.

`Collections.nCopies(n, objeto)`

Gera uma lista imutável contendo n repetições do mesmo objeto. Útil para inicializar um placar de disponibilidade com um valor padrão repetido.

`Collections.addAll(lista, a, b, c)`

Insere vários elementos avulsos de uma só vez, sem precisar empacotá-los antes numa coleção.

`Collections.copy(destino, origem)`

Copia todos os elementos da origem para o destino — o destino precisa ter tamanho maior ou igual ao da origem, senão lança exceção.

`Collections.shuffle(lista)`

Embaralha a ordem dos elementos aleatoriamente — comum em sorteios ou para simular uma seleção de destaques do catálogo.

`Collections.reverse(lista)`

Inverte a ordem atual dos elementos, sem depender de nenhum critério de comparação.

`Collections.binarySearch(lista, alvo)`

Executa busca binária, muito mais rápida que percorrer a lista item a item — mas exige que a lista já esteja ordenada com o mesmo critério usado na busca.

`Collections.frequency(lista, elemento)`

Conta quantas vezes um elemento aparece na coleção.

`Collections.max(lista, comparador) / Collections.min(...)`

Retornam, respectivamente, o maior e o menor elemento segundo o comparador informado.

`Collections.rotate(lista, distancia)`

Desloca os elementos circularmente. Distância positiva empurra os últimos elementos para o início; distância negativa faz o caminho inverso.

`Collections.swap(lista, i, j)`

Troca de posição dois elementos, identificados pelos seus índices.

BINARYSEARCH EXIGE ORDENAÇÃO PRÉVIA

Diferente de `indexOf()`, que funciona em qualquer lista, o `Collections.binarySearch()` só produz resultado confiável se a lista já estiver ordenada com o mesmo critério (`Comparator` ou ordem natural) usado na busca.

02

CAPÍTULO

Set e HashSet

Unicidade, ausência de ordem garantida e operações em tempo constante

Um Set representa um conjunto sem duplicatas: cada elemento só pode aparecer uma vez, e a maioria das implementações não garante nenhuma ordem específica de iteração. A troca que se faz por essa falta de ordem é velocidade — no HashSet, verificar se um elemento existe, adicionar ou remover são operações $O(1)$, ou seja, o tempo de execução praticamente não muda conforme a coleção cresce.

Diferente de uma List, não existe a ideia de "pegar o elemento da posição 3" num Set — a única pergunta que ele responde bem é "esse elemento está aqui ou não?". Isso o torna a escolha natural sempre que o objetivo é garantir unicidade, eliminar repetições ou checar pertencimento rapidamente.

```
Set<String> isbnCadastrados = new HashSet<>();

isbnCadastrados.add("978-85-7522-000-1");

isbnCadastrados.add("978-85-7522-000-2");

isbnCadastrados.add("978-85-7522-000-1"); // ignorado, já existe

isbnCadastrados.size(); // 2
```

Criando Sets imutáveis

Set.of(elemento1, elemento2, ...)

Cria um Set imutável a partir dos elementos informados. Tentar chamar `add()` no resultado lança `UnsupportedOperationException`.

new HashSet<>(setImutavel)

Forma padrão de transformar um Set imutável em um HashSet comum, modificável, quando a aplicação precisar inserir ou remover elementos depois.

```
Set<String> generosBasicos = Set.of("Ficção", "Técnico", "Biografia");

Set<String> generosAtivos = new HashSet<>(generosBasicos);

generosAtivos.add("Infantil"); // agora funciona
```

Como o HashSet funciona por dentro

Internamente, o HashSet delega o trabalho pesado a um HashMap — cada elemento vira uma chave desse mapa interno. Os elementos são distribuídos em posições chamadas buckets, escolhidas a partir do

hashCode() de cada objeto. Dentro do mesmo bucket, o equals() é o método usado para decidir se dois objetos que caíram na mesma posição são, de fato, o mesmo elemento ou apenas uma coincidência de hash.

Operação	Complexidade
add()	O(1)
contains()	O(1)
remove()	O(1)

UM ÚNICO NULL É PERMITIDO

O HashSet aceita no máximo um elemento null. As complexidades O(1) descritas acima pressupõem uma boa distribuição dos hashCodes — muitas colisões concentradas nos mesmos buckets degradam a performance para O(n).

O que acontece ao tentar inserir uma duplicata?

Quando dois objetos são considerados iguais pelo equals(), o HashSet mantém o primeiro que foi inserido e simplesmente ignora as tentativas seguintes — não existe substituição, diferente do que acontece no HashMap, que substitui o valor associado a uma chave repetida. Isso é explorado em detalhe no próximo capítulo, quando falamos do contrato entre hashCode() e equals().

03

CAPÍTULO

hashCode() e equals()

O contrato entre os dois métodos e o funcionamento interno do HashSet

Toda classe em Java herda `equals()` e `hashCode()` de `Object`. Isoladamente eles parecem detalhes técnicos, mas são a base de qualquer estrutura baseada em hashing — `HashSet`, `LinkedHashSet`, `HashMap` e `LinkedHashMap` dependem inteiramente deles para funcionar corretamente.

equals()

A implementação herdada de `Object` compara referências de memória: dois objetos só são considerados iguais se forem exatamente o mesmo objeto na memória. Isso significa que dois livros com o mesmo ISBN, título e autor, mas criados como instâncias diferentes, seriam tratados como diferentes — o que raramente é o comportamento desejado.

Por isso, sobrescrevemos `equals()` para comparar os atributos que realmente definem a identidade lógica do objeto. Para um livro, por exemplo, faz sentido considerar que dois objetos representam o mesmo exemplar quando têm o mesmo ISBN, independente de outros campos.

Estrutura recomendada para o equals()

1. Verificar mesma referência

Se `this == obj`, os dois já são o mesmo objeto — retorna `true` imediatamente, sem comparar campo a campo.

2. Verificar o tipo

Se `obj` não é do mesmo tipo (checado com `instanceof`), retorna `false` — comparar campos de tipos diferentes não faz sentido.

3. Comparar os atributos relevantes

Compara apenas os campos que definem a identidade lógica do objeto — no exemplo do livro, o ISBN.

hashCode()

O `hashCode()` devolve um número inteiro usado pelas estruturas de hashing para localizar um objeto rapidamente. Em vez de comparar o alvo com cada elemento da coleção — o que seria $O(n)$ — o Java calcula esse número, vai direto ao bucket correspondente e só então usa o `equals()` para confirmar a igualdade dentro daquele bucket específico.

As regras do contrato

1. equals() true implica hashCode() igual

Se `a.equals(b)` é `true`, então `a.hashCode()` precisa, obrigatoriamente, ser igual a `b.hashCode()`. Quebrar essa regra é a causa mais comum de "duplicatas fantasma" em Sets: dois objetos logicamente iguais caem em buckets diferentes e o Set não percebe a repetição.

2. Consistência

O mesmo objeto deve produzir sempre o mesmo hash, contanto que os campos usados no cálculo não tenham sido alterados.

3. Eficiência

O cálculo do hash deve ser rápido — de preferência baseado em campos imutáveis, para evitar resultados que mudam ao longo do tempo.

4. Colisões são permitidas

Dois hashes iguais não implicam objetos iguais — isso se chama colisão, e é justamente o `equals()` quem resolve a ambiguidade quando ela acontece.

SOBRESCREVER UM SEM O OUTRO QUEBRA O CONTRATO

Se você sobrescrever `equals()` sem sobrescrever `hashCode()` (ou vice-versa), o Set passa a aceitar "duplicatas" — porque objetos considerados iguais pelo `equals()` podem cair em buckets diferentes, já que o `hashCode()` herdado de `Object` continua baseado na referência de memória.

Por que a constante 31 aparece tanto?

É comum ver a expressão `resultado = 31 * resultado + campoHash` em implementações manuais de `hashCode()`. O número 31 é primo, o que ajuda a espalhar melhor os hashes e reduzir colisões — e, como vantagem extra, a multiplicação por 31 pode ser otimizada internamente pela JVM como uma operação de deslocamento de bits, mais rápida que uma multiplicação comum.

Implementação prática com Objects

A forma mais segura de implementar os dois métodos hoje em dia é combinar `Objects.equals()` e `Objects.hash()`, que já tratam valores null automaticamente e aceitam múltiplos campos.

```
public class Livro {

    private String isbn;

    private String titulo;

    // equals e hashCode SEMPRE sobrescritos juntos:

    @Override

    public boolean equals(Object o) {

        if (this == o) return true;

        if (!(o instanceof Livro outro)) return false;

        return Objects.equals(isbn, outro.isbn);

    }

    @Override

    public int hashCode() {

        return Objects.hash(isbn); // baseado no mesmo campo usado no equals

    }

}
```

RECORDS FAZEM ISSO AUTOMATICAMENTE

Use `Objects.hash(campo1, campo2, ...)` quando a identidade do objeto depender de mais de um campo. O IntelliJ também gera os dois métodos automaticamente (Alt+Insert). E se a classe puder ser modelada como record, `equals()` e `hashCode()` já saem prontos, coerentes entre si.

O caminho de uma inserção no HashSet

Ao inserir um elemento, o HashSet segue uma sequência fixa: calcula o `hashCode()` do objeto para descobrir o bucket → verifica se já existe algum elemento ali → se existir, chama `equals()` para confirmar se é realmente o mesmo elemento → sendo iguais, a inserção é ignorada; sendo diferentes (uma colisão de hash), o novo elemento é adicionado ao mesmo bucket. Em resumo: o `hashCode()` encontra a região certa rapidamente, e o `equals()` confirma a igualdade de verdade.

04

CAPÍTULO

LinkedHashSet, TreeSet e NavigableSet

Ordem de inserção, ordenação automática e navegação por proximidade

LinkedHashSet — ordem de inserção preservada

O `LinkedHashSet` une a velocidade do `HashSet` a uma iteração previsível: os elementos aparecem sempre na ordem em que foram inseridos. Isso é possível porque, além da tabela hash usada para as buscas rápidas, ele mantém internamente uma lista duplamente ligada só para registrar a sequência de inserção. A performance permanece igual à do `HashSet` — $O(1)$ para `add`, `contains` e `remove` — porque a lista ligada só organiza a ordem de iteração, sem interferir na busca.

```
Set<String> historicoConsulta = new LinkedHashSet<>();

historicoConsulta.add("Clean Code");

historicoConsulta.add("Effective Java");

historicoConsulta.add("Clean Code"); // ignorado, já existe

// Iteração: Clean Code, Effective Java (ordem de inserção)
```

TreeSet — ordenação automática

O `TreeSet` mantém os elementos sempre ordenados — pela ordem natural do tipo (via `Comparable`) ou por um `Comparator` fornecido no construtor. Por baixo dos panos, ele é implementado como uma árvore rubro-negra (Red-Black Tree), uma árvore binária de busca que se rebalancia automaticamente a cada inserção ou remoção, mantendo elementos menores à esquerda e maiores à direita. Essa organização custa um pouco de performance em troca da ordenação: $O(\log n)$ em vez do $O(1)$ do `HashSet`.

Operação	Complexidade
<code>add()</code>	$O(\log n)$
<code>contains()</code>	$O(\log n)$
<code>remove()</code>	$O(\log n)$

Comparable x Comparator

Existem duas formas de dizer ao `TreeSet` como ordenar seus elementos: `Comparable` define a ordem natural do próprio tipo — a classe do elemento sabe comparar-se a outro de sua espécie. `Comparator` define uma ordenação externa, decidida por outra classe e passada diretamente ao construtor do `TreeSet`, útil quando o mesmo tipo precisa de critérios de ordenação diferentes em contextos diferentes.

```
TreeSet<Integer> anosDisponiveis = new TreeSet<>(Set.of(1998, 2019, 2005, 2022));

// Iteração: 1998, 2005, 2019, 2022 — ordem natural dos Integer
```

NavigableSet — navegando por proximidade

O `TreeSet` implementa a interface `NavigableSet`, que adiciona métodos capazes de responder perguntas como "qual o valor mais próximo, para cima ou para baixo, de um determinado ponto?". Considerando o conjunto de anos {1998, 2005, 2019, 2022} como referência:

floor(x)

Retorna o maior elemento menor ou igual a x. `floor(2020)` devolve 2019.

lower(x)

Retorna o maior elemento estritamente menor que x. `lower(2019)` devolve 2005.

ceiling(x)

Retorna o menor elemento maior ou igual a x. `ceiling(2000)` devolve 2005.

higher(x)

Retorna o menor elemento estritamente maior que x. `higher(2005)` devolve 2019.

first() / last()

Retornam, respectivamente, o menor e o maior elemento do conjunto — 1998 e 2022 no exemplo.

headSet(x) / tailSet(x)

`headSet(x)` retorna todos os elementos menores que x (exclusivo); `tailSet(x)` retorna todos os elementos maiores ou iguais a x (inclusivo).

pollFirst() / pollLast()

Removem e retornam, respectivamente, o menor e o maior elemento do conjunto — únicos métodos de navegação que alteram a coleção.

descendingSet()

Retorna uma view do mesmo conjunto em ordem decrescente.

```
TreeSet<Integer> anos = new TreeSet<>(Set.of(1998, 2005, 2019, 2022));

anos.floor(2020); // 2019 (maior valor <= 2020)

anos.ceiling(2000); // 2005 (menor valor >= 2000)

anos.headSet(2019); // [1998, 2005] (exclusivo)

anos.tailSet(2019); // [2019, 2022] (inclusivo)
```

HEADSET, TAILSET E SUBSET RETORNAM VIEWS

O resultado desses três métodos não é uma cópia — é uma janela viva para dentro do TreeSet original. Alterar a view reflete no conjunto original, e vice-versa.

05

CAPÍTULO

Operações de Set

União, interseção, diferença e diferença simétrica entre coleções

Java não tem métodos chamados literalmente `union()` ou `intersection()`, mas as operações matemáticas clássicas de conjuntos podem ser reproduzidas com três métodos herdados de `Collection`: `addAll()`, `retainAll()` e `removeAll()`. O detalhe importante é que os três modificam o `Set` em que são chamados — então, sempre que o conjunto original precisar ser preservado, o primeiro passo é trabalhar sobre uma cópia.

ESSAS OPERAÇÕES ALTERAM O SET ORIGINAL

Sempre construa uma cópia com `new HashSet<>(conjuntoOriginal)` antes de aplicar `addAll`, `retainAll` ou `removeAll`, caso precise manter o conjunto original intacto. Para apenas verificar relações sem alterar nada, use `containsAll()` (subconjunto) ou `equals()` (igualdade).

addAll() — União**`uniao.addAll(outroConjunto)`**

Acrescenta ao `Set` atual todos os elementos do outro conjunto, sem gerar duplicatas — o resultado é a soma dos dois grupos. É uma operação simétrica: $A \cup B$ é sempre igual a $B \cup A$.

```
Set<String> acervoFilialCentro = new HashSet<>(Set.of("Dom Casmurro", "1984", "Sapiens"));

Set<String> acervoFilialSul = Set.of("1984", "Sapiens", "O Hobbit");

Set<String> catalogoCombinado = new HashSet<>(acervoFilialCentro);

catalogoCombinado.addAll(acervoFilialSul);

// {Dom Casmurro, 1984, Sapiens, O Hobbit}
```

retainAll() — Interseção**`intersecao.retainAll(outroConjunto)`**

Mantém apenas os elementos que também estão presentes no outro conjunto, removendo os que não têm correspondência — o resultado é o que existe nos dois grupos simultaneamente. Também é simétrica: $A \cap B$ é igual a $B \cap A$.


```
Set<String> emComum = new HashSet<>(acervoFilialCentro);

emComum.retainAll(acervoFilialSul);

// {1984, Sapiens}
```

removeAll() — Diferença

diferenca.removeAll(outroConjunto)

Remove do Set atual qualquer elemento que também exista no outro conjunto — o resultado são apenas os elementos exclusivos do primeiro grupo. Ao contrário das duas anteriores, essa operação é assimétrica: $A - B$ normalmente não é igual a $B - A$.

```
Set<String> soNoCentro = new HashSet<>(acervoFilialCentro);

soNoCentro.removeAll(acervoFilialSul);

// {Dom Casmorro} (apenas o que está só na filial Centro)
```

Diferença simétrica

A diferença simétrica reúne os elementos que pertencem exclusivamente a cada um dos conjuntos — tudo, exceto a interseção. Ela pode ser obtida de duas formas equivalentes: subtraindo cada conjunto do outro e unindo os resultados, ou unindo os dois conjuntos e removendo depois a interseção.

```
Set<String> soCentro = new HashSet<>(acervoFilialCentro);

soCentro.removeAll(acervoFilialSul); // {Dom Casmurro}

Set<String> soSul = new HashSet<>(acervoFilialSul);

soSul.removeAll(acervoFilialCentro); // {O Hobbit}

soCentro.addAll(soSul);

// diferença simétrica: {Dom Casmurro, O Hobbit}
```

Verificações sem alterar a coleção

`conjunto.containsAll(outroConjunto)`

Confirma se todos os elementos de outro conjunto já estão contidos no atual, sem modificar nada — útil para checar se um pedido de reserva pode ser totalmente atendido por um acervo.

`conjuntoA.equals(conjuntoB)`

Retorna true quando os dois Sets têm exatamente os mesmos elementos, independentemente da ordem interna de armazenamento.

Operação	Método Java	Quando usar
União	<code>addAll()</code>	Consolidar dados de duas fontes sem gerar repetições
Interseção	<code>retainAll()</code>	Encontrar registros presentes nos dois conjuntos
Diferença	<code>removeAll()</code>	Isolar o que é exclusivo de um dos conjuntos
Dif. simétrica	<code>removeAll()</code> + <code>addAll()</code>	Descobrir o que mudou entre duas versões
Subconjunto	<code>containsAll()</code>	Verificar se um pedido pode ser totalmente atendido
Igualdade	<code>equals()</code>	Verificar se dois conjuntos têm exatamente os mesmos itens

06

CAPÍTULO

Map, HashMap e Views

Pares chave-valor, métodos modernos do JDK 8+ e as visões do Map

A interface Map guarda pares chave → valor. Diferente de List ou Set, todo registro num Map carrega duas informações: a chave, que identifica o item de forma única, e o valor, o dado associado a ela. Chaves nunca se repetem dentro do mesmo Map, mas o mesmo valor pode estar associado a chaves diferentes sem problema.

HashMap — a implementação mais comum

O HashMap usa a mesma estratégia de hashing do HashSet — de fato, é a estrutura que o HashSet usa internamente. Cada entrada é guardada como um nó contendo chave, valor e hash, distribuído em buckets pelo hash da chave. Não há garantia de ordem de iteração, mas as operações básicas são $O(1)$.

Operação	Complexidade
put()	$O(1)$
get()	$O(1)$
remove()	$O(1)$
containsKey()	$O(1)$

```
Map<String, Livro> catalogo = new HashMap<>();

catalogo.put("978-85-0001", new Livro("Sapiens", "Yuval Harari"));

catalogo.put("978-85-0002", new Livro("1984", "George Orwell"));

catalogo.get("978-85-0001"); // devolve o objeto Livro correspondente

catalogo.containsKey("978-85-0002"); // true
```

Métodos básicos

put(chave, valor)

Insere um novo par ou substitui o valor de uma chave já existente. Retorna null se a chave era nova, ou o valor anterior se ela já existia.

get(chave)

Recupera o valor associado à chave informada, retornando null se ela não existir no mapa.

remove(chave)

Remove a entrada correspondente e devolve o valor que estava associado a ela.

containsKey(chave) / containsValue(valor)

Verificam, respectivamente, se uma chave ou um valor existem no mapa — `containsKey()` é $O(1)$, enquanto `containsValue()` precisa percorrer as entradas, sendo $O(n)$.

Métodos modernos (JDK 8+)

A partir do Java 8, o Map ganhou métodos que evitam blocos inteiros de verificação manual que antes eram necessários para tarefas comuns, como inicializar um valor só se ele ainda não existir, ou combinar valores repetidos.

putIfAbsent(chave, valor)

Insere o valor apenas se a chave ainda não existir no mapa — nunca sobrescreve uma entrada já presente. Retorna o valor existente quando a chave já estava lá, ou null quando a inserção foi feita.

```
catalogo.putIfAbsent("978-85-0001", new Livro("Outro título", "Outro Autor"));

// ignorado — a chave já existia, o livro original é mantido
```

merge(chave, valor, funcaoDeCombinacao)

Se a chave não existe, insere o valor diretamente. Se já existe, aplica a função informada sobre o valor atual e o novo valor, e guarda o resultado. É a forma mais direta de acumular contagens ou mesclar dados de registros repetidos.

```
Map<String, Integer> emprestimosPorGenero = new HashMap<>();

emprestimosPorGenero.merge("Ficção", 1, Integer::sum); // cria: Ficção -> 1

emprestimosPorGenero.merge("Ficção", 1, Integer::sum); // soma: Ficção -> 2
```

compute(chave, funcao)

Recalcula o valor da chave sempre — exista ela ou não, a função recebe o valor atual (ou null) e devolve o novo valor.

computeIfAbsent(chave, funcao)

Executa a função apenas quando a chave ainda não existe — muito usado para inicializar estruturas aninhadas, como uma lista dentro do mapa, sem precisar checar antes se ela já foi criada.

computeIfPresent(chave, funcao)

Executa a função apenas quando a chave já existe, transformando o valor atual.

```
Map<String, List<String>> emprestimosPorMembro = new HashMap<>();

emprestimosPorMembro.computeIfAbsent("Ana", k -> new ArrayList<>()).add("Sapiens");

// cria a lista automaticamente na primeira vez, e só adiciona nas seguintes
```

replace(chave, novoValor)

Substitui o valor apenas se a chave já existir no mapa.

replace(chave, valorEsperado, novoValor)

Substitui o valor somente se o valor atual for igual ao esperado — uma forma segura de atualizar apenas quando o estado é o previsto.

replaceAll(funcao)

Percorre todas as entradas do mapa e atualiza cada uma com o resultado da função, que recebe a chave e o valor atuais.

As views do Map: keySet, values e entrySet

As três views abaixo não são cópias — são janelas vivas para dentro do próprio Map: qualquer alteração feita nelas reflete no mapa original, e vice-versa.

View	Tipo retornado	Duplicatas?	Permite add/addAll?
keySet()	Set	Não	Não — lança exceção
values()	Collection	Sim	Não — lança exceção
entrySet()	Set<	Não	Não — lança exceção

ENTRYSET() É A VIEW MAIS COMPLETA

keySet() não aceita add()/addAll() porque uma chave isolada não tem valor associado — isso geraria uma inconsistência. Mas remove(), removeAll(), retainAll() e clear() são permitidos e refletem diretamente no Map. Já entrySet() expõe cada Map.Entry com getKey(), getValue() e setValue(), permitindo iterar e alterar valores sem precisar acessar o mapa de novo.

```
for (Map.Entry<String, Livro> item : catalogo.entrySet()) {

    System.out.println(item.getKey() + " -> " + item.getValue());

}

// forma mais concisa com forEach:

catalogo.forEach((isbn, livro) -> System.out.println(isbn + ": " + livro));

// remover via keySet também altera o Map original:

catalogo.keySet().removeIf(isbn -> isbn.startsWith("978-85-000"));
```

07

CAPÍTULO

LinkedHashMap, TreeMap e NavigableMap

Ordem de inserção, cache LRU, chaves ordenadas e navegação por intervalo

LinkedHashMap — ordem de inserção

O LinkedHashMap combina um HashMap com uma lista duplamente ligada extra, responsável apenas por registrar a ordem de inserção das entradas. O hashing garante a velocidade nas buscas; a lista ligada garante que a iteração siga sempre a mesma sequência em que os dados foram inseridos — diferente do HashMap comum, que não garante nenhuma ordem.

```
Map<String, String> visitasRecentes = new LinkedHashMap<>();

visitasRecentes.put("Sapiens", "12/03");

visitasRecentes.put("1984", "14/03");

visitasRecentes.put("O Hobbit", "15/03");

// Iteração respeita a ordem de inserção: Sapiens, 1984, O Hobbit
```

Access order — ordenando pelo uso

Um construtor especial do LinkedHashMap troca o comportamento padrão (ordem de inserção) por ordem de acesso: toda vez que uma chave é lida com get() ou reescrita com put(), ela é movida para o final da fila interna — o item mais recentemente usado passa a ficar sempre por último.

```
// true = ordena por acesso; false (padrão) = ordena por inserção

Map<String, String> capasVisualizadas = new LinkedHashMap<>(16, 0.75f, true);

capasVisualizadas.put("Sapiens", "capa1.jpg");

capasVisualizadas.put("1984", "capa2.jpg");

capasVisualizadas.put("O Hobbit", "capa3.jpg");

capasVisualizadas.get("Sapiens");

// ordem interna agora: 1984, O Hobbit, Sapiens (Sapiens foi ao final)
```

removeEldestEntry() — construindo um cache LRU

Sobrescrevendo `removeEldestEntry()`, é possível transformar um `LinkedHashMap` num cache LRU (Least Recently Used, ou "menos recentemente usado"), que descarta automaticamente o item mais antigo assim que um limite de tamanho é ultrapassado. É uma das aplicações mais práticas dessa estrutura em sistemas reais — por exemplo, guardar em memória apenas as capas de livros consultadas mais recentemente.

```
// Cache que guarda no máximo as 50 capas mais recentemente vistas:

Map<String, String> cacheCapas = new LinkedHashMap<>(16, 0.75f, true) {

    @Override

    protected boolean removeEldestEntry(Map.Entry<String, String> maisAntiga) {

        return size() > 50; // remove a entrada mais antiga ao ultrapassar 50

    }

};
```

REMOVEELDESTENTRY RODA AUTOMATICAMENTE

Esse método é chamado pelo próprio `LinkedHashMap` logo após cada `put()`. Retornar `true` remove a entrada mais antiga; retornar `false` a mantém.

TreeMap — chaves ordenadas

O `TreeMap` mantém as chaves sempre ordenadas, seguindo os mesmos princípios do `TreeSet`: internamente é uma árvore rubro-negra auto-balanceada, sem uso de hashing. As chaves precisam implementar `Comparable`, ou o `TreeMap` precisa receber um `Comparator` no construtor.

Operação	Complexidade
<code>put()</code>	$O(\log n)$
<code>get()</code>	$O(\log n)$
<code>remove()</code>	$O(\log n)$

SortedMap — métodos de intervalo

`firstKey()` / `lastKey()`

Retornam, respectivamente, a menor e a maior chave presentes no mapa.

`headMap(chave)`

Retorna todas as entradas cuja chave é menor que a informada (exclusivo).

`tailMap(chave)`

Retorna todas as entradas cuja chave é maior ou igual à informada (inclusivo).

subMap(chaveInicial, chaveFinal)

Retorna as entradas entre as duas chaves — a inicial incluída, a final excluída.

NavigableMap — navegação por proximidade

Assim como o NavigableSet faz com valores, o NavigableMap permite localizar chaves vizinhas de um ponto de referência, em duas variações: uma que devolve só a chave, outra que devolve a entrada completa (chave + valor).

lowerKey(k) / lowerEntry(k)

Maior chave estritamente menor que k.

floorKey(k) / floorEntry(k)

Maior chave menor ou igual a k.

ceilingKey(k) / ceilingEntry(k)

Menor chave maior ou igual a k.

higherKey(k) / higherEntry(k)

Menor chave estritamente maior que k.

firstEntry() / lastEntry()

Retornam a entrada completa (chave + valor) do menor e do maior elemento, respectivamente.

pollFirstEntry() / pollLastEntry()

Removem e retornam, respectivamente, a primeira e a última entrada — os únicos métodos de navegação que alteram o mapa.

descendingKeySet() / descendingMap()

Retornam, respectivamente, as chaves em ordem decrescente e uma view completa do mapa invertida.

```
TreeMap<Integer, String> lancamentosPorAno = new TreeMap<>();

lancamentosPorAno.put(1998, "Harry Potter");

lancamentosPorAno.put(2005, "O Código Da Vinci");

lancamentosPorAno.put(2019, "Sapiens");


lancamentosPorAno.floorKey(2010); // 2005 (maior chave <= 2010)

lancamentosPorAno.ceilingKey(2010); // 2019 (menor chave >= 2010)

lancamentosPorAno.headMap(2019); // {1998=..., 2005=...}

lancamentosPorAno.tailMap(2019); // {2019=...}

lancamentosPorAno.pollFirstEntry(); // remove e retorna 1998=Harry Potter
```

HEADMAP, TAILMAP E SUBMAP TAMBÉM RETORNAM VIEWS

O mesmo aviso do TreeSet vale aqui: essas três views não são cópias. Alterações nelas refletem no TreeMap original e vice-versa.

Comparando as três implementações de Map

Característica	HashMap	LinkedHashMap	TreeMap
Ordenação	Nenhuma	Inserção	Ordenada
Usa hashing	Sim	Sim	Não
Estrutura em árvore	Não	Não	Sim
Serve para cache LRU	Não	Sim	Não
Navegação por intervalo	Não	Não	Sim
put() / get() / remove()	O(1)	O(1)	O(log n)

08

CAPÍTULO

EnumSet e EnumMap

Coleções especializadas para enums e representação por vetores de bits

Para trabalhar com constantes de um enum, o Java oferece duas implementações especializadas: EnumSet e EnumMap. Elas existem porque um enum já tem duas garantias que HashSet e HashMap não têm de graça: uma quantidade fixa e conhecida de constantes, e uma ordem de declaração fixa. Aproveitando essas garantias, ambas as estruturas conseguem ser muito mais compactas e rápidas que suas equivalentes genéricas.

EnumSet — conjuntos representados por bits

O EnumSet é uma implementação de Set feita exclusivamente para armazenar constantes de um único tipo enum. É uma classe abstrata — não pode ser instanciada diretamente com new; em vez disso, é sempre criada através de métodos de fábrica estáticos.

Internamente, cada constante do enum tem um ordinal() — sua posição de declaração — e o EnumSet guarda quais posições estão presentes usando bits individuais dentro de um número. Um enum de gêneros como FICCAO, TECNICO, BIOGRAFIA, INFANTIL e POESIA (ordinais 0 a 4) representando o conjunto {FICCAO, BIOGRAFIA, POESIA} ficaria assim, internamente:

```
// posições: 0 1 2 3 4

// gêneros: FICCAO TECNICO BIOGRAFIA INFANTIL POESIA

// bits: 1 0 1 0 1

// presente presente presente
```

Essa representação em bits torna operações de conjunto inteiras — containsAll, retainAll, removeAll — executáveis em $O(1)$ através de matemática binária, em vez de precisar percorrer elemento por elemento.

Variante	Capacidade	Armazenamento	Complexidade
RegularEnumSet	Até 64 constantes	Um único long (64 bits)	$O(1)$
JumboEnumSet	Mais de 64 constantes	Array de longs	$O(n/64)$

Métodos de fábrica

EnumSet.of(c1, c2, ...)

Cria um EnumSet contendo exatamente as constantes informadas.

EnumSet.allOf(Enum.class)

Cria um EnumSet com todas as constantes do enum informado.

EnumSet.noneOf(Enum.class)

Cria um EnumSet vazio, mas já vinculado ao tipo do enum informado.

EnumSet.range(inicio, fim)

Cria um EnumSet com todas as constantes entre duas posições, ambas incluídas.

EnumSet.copyOf(colecao)

Cria um EnumSet a partir de qualquer Collection de constantes do mesmo enum.

EnumSet.complementOf(outroSet)

Cria um EnumSet com todas as constantes que NÃO estão presentes no conjunto informado — equivalente a pegar allOf() e remover o que já está no outro set.

```
enum Genero { FICCAO, TECNICO, BIOGRAFIA, INFANTIL, POESIA }

EnumSet<Genero> generosInfantis = EnumSet.of(Genero.INFANTIL, Genero.POESIA);

EnumSet<Genero> primeirosTres = EnumSet.range(Genero.FICCAO, Genero.BIOGRAFIA);

EnumSet<Genero> todos = EnumSet.allOf(Genero.class);

// complementOf equivale a todos.removeAll(generosInfantis):

EnumSet<Genero> naoInfantis = EnumSet.complementOf(generosInfantis);

// {FICCAO, TECNICO, BIOGRAFIA}
```

A ORDEM SEMPRE SEGUE A DECLARAÇÃO DO ENUM

Um EnumSet criado com EnumSet.of(Genero.POESIA, Genero.FICCAO, Genero.TECNICO) sempre itera na ordem de declaração do enum — FICCAO, TECNICO, POESIA — nunca na ordem em que os elementos foram passados. Não é necessário TreeSet, Comparable ou Comparator para isso.

EnumMap — mapas com chaves de enum

O EnumMap é a versão especializada de Map para chaves do tipo enum. Diferente do EnumSet, não é abstrato — pode ser instanciado com new, mas exige que o tipo do enum seja informado no construtor. As chaves são sempre ordenadas pelo ordinal() da constante, independentemente da ordem em que foram inseridas.

```
EnumMap<Genero, Integer> emprestimosPorGenero = new EnumMap<>(Genero.class);

emprestimosPorGenero.put(Genero.POESIA, 4);

emprestimosPorGenero.put(Genero.FICCAO, 21);

emprestimosPorGenero.put(Genero.TECNICO, 9);

// Iteração sempre segue a ordem do enum, não a de inserção:

// FICCAO -> 21, TECNICO -> 9, POESIA -> 4
```

Todos os métodos vistos no `HashMap` continuam disponíveis normalmente — `getOrDefault`, `putIfAbsent`, `computeIfAbsent`, `merge` — só que operando sobre chaves de `enum`, com a vantagem extra da ordenação automática e menor consumo de memória.

Operação	Complexidade
<code>put()</code>	$O(1)$
<code>get()</code>	$O(1)$
<code>remove()</code>	$O(1)$
<code>containsKey()</code>	$O(1)$

EnumSet/EnumMap comparados às versões genéricas

Característica	HashSet	EnumSet
Especializado para enum	Não	Sim
Ordenação automática	Não	Sim (ordem do enum)
Representação em bits	Não	Sim
Operações em massa	$O(n)$	$O(1)$
Consumo de memória	Maior	Menor

Característica	HashMap	EnumMap
Especializado para enum	Não	Sim
Ordenação automática	Não	Sim (ordem do enum)
Usa hashing tradicional	Sim	Não
Consumo de memória	Maior	Menor

QUANDO OS DADOS SÃO UM ENUM, APROVEITE

Não existe implementação "melhor" para todo cenário. Sempre que os dados forem naturalmente representáveis como um enum, EnumSet e EnumMap tendem a ser mais rápidos, mais compactos e mais expressivos que HashSet e HashMap tradicionais.

Resumo Final

Com List, Set, Map, NavigableSet/Map e as coleções especializadas de Enum reunidos num só lugar, fica mais fácil escolher a estrutura certa para cada problema — a diferença entre um código apenas funcional e um código eficiente costuma estar justamente nessa escolha.

Estrutura	Quando escolher
HashSet	Unicidade com máxima velocidade — não precisa de ordem
LinkedHashSet	Unicidade com ordem de inserção previsível
TreeSet	Elementos sempre ordenados, com navegação por proximidade
EnumSet	Conjuntos de constantes enum, com operações em massa $O(1)$
HashMap	Pares chave-valor com máxima velocidade — sem ordem garantida
LinkedHashMap	HashMap com ordem de inserção — base para caches LRU
TreeMap	Chaves sempre ordenadas, com navegação por intervalo
EnumMap	Mapas com chaves enum — ordenação automática, menor memória